

Codex Web 工作台 v0.2：面向不可靠网页通道的工具型 Agent 桥接、会话恢复与状态管理

3099404236^{1*}

¹Independent developer

本报告记录 Codex Web 工作台的第二个公开版本。系统保留 Codex CLI 的文件、终端、补丁、Skills 与 MCP 工具循环，同时把主模型请求临时转发到用户已经登录的 ChatGPT 网页。v0.2 从初代最小 Responses 兼容桥接器扩展为可恢复的本地工作台，新增已有 Chrome 标签页复用、持久化任务到网页对话 URL 的映射、同页网关恢复、长文本分块、中间 Thinking 状态过滤、完成请求去重、对话确实失效后的结构化检查点、工作区任务面板、人工阶段门控和可选微任务委托。实现借鉴 Responses API 的函数调用、会话链、compaction 与 persisted reasoning 思路，但不声称获得或复制官方内部推理状态。2026-08-11 的本地验证包含 52 项自动测试，全部通过。公开版本不包含 Cookie、真实对话、API key、浏览器 profile、运行日志或本机私用 ZIP。

Codex | Responses API | ChatGPT Web | agent state | browser automation | fault recovery

 <https://github.com/3099404236/chatgpt-web-codex-bridge>

版本定位

Codex 的代理能力并不等同于一次模型回答。外层客户端负责声明工具、执行本机命令与补丁、保存任务状态，再把工具结果送回模型。初代版本验证了一个最小命题：即使主模型来自可见网页，只要外层工具循环仍由 Codex 控制，网页模型仍可以参与连续的“读取、修改、测试、再修正”。

第二版面对的是另一个更实际的问题：网页连接会断、页面会卡在 Thinking、Cloudflare 会返回 504、终端会关闭、浏览器会刷新，而且网页没有原生 previous_response_id 或可导出的 reasoning item。系统如何避免每次新开对话和重放全部历史，又能在确有必要时恢复任务？

本项目仍是本机实验工具，不是 OpenAI 官方 API，也不会把 ChatGPT 订阅转换成 API 额度。它不绕过登录、验证码、用量限制或网站风控。

设计目标与非目标

v0.2 的目标可归纳为五点：

- 工具所有权不变：文件和终端操作仍由 Codex 在本机执行；
- 对话优先复用：普通失败不放弃旧 URL，不因轮数或字符数主动新建网页对话；
- 终态明确：每个流式请求最终产生 response.completed 或 response.failed；
- 恢复可审计：确需换对话时传递结构化检查点，而不是不可检查的完整历史拼接；
- 本机数据隔离：公开代码与 Cookie、任务历史、日志、密钥和浏览器 profile 分离。

非目标同样重要：当前实现不追求覆盖完整 Responses API，不提供 OpenAI 托管工具，不读取 ChatGPT 内部隐藏推理，不保证网页 DOM 永久稳定，也不把浏览器自动化包装成生产级服务。

从官方接口思想到网页替代实现

官方 Responses 会话链可以使用 previous_response_id 连接前后响应，函数调用流程则要求把模型输出的调用与对应 function_call_output 一起送入后续请求^{1,2}。网页桥没有这些服务器端对象，因此必须为每个概念寻找外部替代物。

OpenAI 官方文档说明，compaction 返回的窗口可用较少 token 携带关键先前状态和推理，且其中的 compaction item 是不透明、非面向人类阅读的对象³；persisted reasoning 则服务于推理连续性，但不会暴露模型的原始推理文本⁴。本项目因此把“可见对话状态”和“隐藏推理状态”明确分开：前者可以通过 URL、消息、工具结果和检查点管理，后者不能从网页可靠获得。

版本	定位	主要能力
v0.1	最小桥接器	Responses JSON/SSE、同对话增量续轮、shell 与 apply_patch 协议、一次网页错误重试、30 项测试。
v0.2	可恢复工作台	已有 Chrome 复用、URL 持久化、同页故障恢复、检查点、去重、长文本通道、工作流面板、可选委托、52 项测试。

表 1 两个公开版本的功能边界。v0.2 保留 v0.1，不替换其公开 PDF。

官方概念	本项目替代物	差异
Responses 会话链	Codex 任务键到 ChatGPT URL 的持久映射	只能依赖网页可见对话和本地记录，不能获得服务器端 response 对象。
函数调用项	网页输出中的严格 YAML/JSON 工具块	需要解析、白名单校验和字符串修复；模型格式波动仍可能导致重试。
函数输出项	Codex 下一轮请求中的工具结果增量	实际执行者仍是 Codex，网页只决定下一步。
Compaction item	本地结构化检查点与最近增量	官方 compaction 可携带不透明状态；本地检查点只保存可解释的外显事实。
Persisted reasoning	无等价物	网页层拿不到原生 reasoning item，不能宣称恢复隐藏推理连续性。

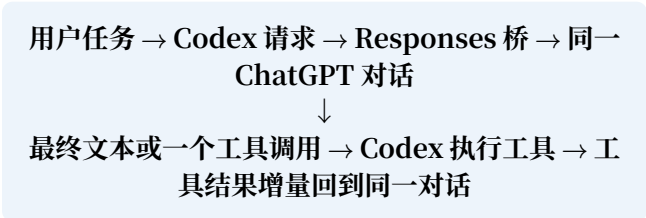
表 2 官方接口概念与黑盒网页条件下的工程替代。

组件	本机端口	职责
Codex TUI	-	组织 Agent 循环，声明工具，执行文件、终端和补丁操作。
App Server	8771 / WS	提供任务、命令、计划、补丁和 MCP 事件。
事件代理	8772 / WS	透明转发 App Server 连接，并把事件副本写入工作台。
Supervisor	8770 / HTTP	持久化工作区、任务、阶段、事件与 OMP 工人状态，并提供网页面板。
Responses 桥	8766 / HTTP	接收 JSON/SSE 请求，维护会话映射、请求缓存、检查点和错误终态。
Chrome DevTools MCP	stdio	列出页面、导航、读取快照、输入提示和观察完成状态。
Browser Relay	9224 / WS/HTTP	官方自动连接在本机失败时，把 chrome.debugger CDP 消息转发给 MCP。
ChatGPT 标签页	远程网页	提供模型生成界面；登录、搜索能力和服务状态均由网页产品自身管理。

表 3 v0.2 的本机组件和职责。所有自建网络服务默认绑定 127.0.0.1。

系统架构

一次正常工具循环如下：



桥接器不会代替 Codex 直接执行网页提出的命令。它只把允许的工具调用转换为 Responses 事件；Codex 执行以后，下一次请求携带结果，网页模型再决定是否继续。单次用户输入之所以能产生多轮操作，循环控制器是 Codex，而不是浏览器脚本。

会话身份、恢复与检查点

桥接器根据 Codex 请求构造稳定任务键，并在本机保存任务键、ChatGPT URL、已发送输入指纹、最后完成结果、失败状态和检查点。恢复顺序为：

- 1. /resume 让 Codex 恢复自己的旧任务；
- 2. 新请求命中持久任务键；

检查点字段	内容	边界
原始目标	任务最初要达到的结果	不是模型隐藏推理。
最近增量	最近用户输入、工具调用与工具结果	数量受限，避免完整历史重放。
最后完成输出	最近一次已确认完成的网页回答	中间 Thinking 不进入此字段。
同步位置	输入指纹、URL 和更新时间	用于判定下一轮应发送什么。
恢复说明	旧 URL 与替代对话的交接原因	让新网页对话知道这是恢复，不是新任务。

表 4 本地检查点是可解释的任务状态，不是官方 compaction item。

- 3. 桥接器导航回已保存的 ChatGPT URL；
- 4. 只发送尚未同步的最近增量；
- 5. 成功后更新同步位置和检查点。

普通超时、断流、Thinking、网关错误、标签页关闭和 DevTools 断线都被视为暂态失败，旧 URL 会继续保留。只有页面连续三次明确报告对话不存在、已删除、不可用或无权访问，才允许建立替代对话。

如果同一个已经完成的 Responses 请求再次到达，桥接器直接返回本地缓存结果。这一设计处理了断流后客户端重试的副作用风险：请求结果未知时自动再次点击发送，可能导致同一句话进入网页两次，随后又重复执行工具。

浏览器复用与 Chrome 连接踩坑

用户希望复用日常 Chrome 中已经打开并登录的 ChatGPT 标签页，而不是每次启动独立 profile。项目首先使用 Chrome DevTools MCP 的 --autoConnect 路径。本机曾出现端口存在、权限开关已启用，但握手无法完成的现象，与 issue 2283 的报告相似⁵。

后续讨论澄清了一个重要误区：在 chrome://inspect/#remote-debugging 的权限代理模式下，/json/version

返回 404 本身是预期现象，MCP 使用的是受授权 WebSocket 路径，不能仅凭 404 判断自动连接必然损坏。问题应被描述为“特定机器上的自动连接握手失败”，而不是“Chrome 150/151 默认 profile 普遍不可用”。

本项目保留两条路径：

- 官方优先：Chrome DevTools MCP 直接连接已授权的 Chrome；
- 本机 fallback：一次性加载的扩展使用 `chrome.debugger`，把 CDP 消息转发到 127.0.0.1:9224，再由 MCP 操作真实标签页。

Relay 只监听本机，不复制 Cookie，也不把 OMP 设为主模型。它仍拥有读取和操作被附加标签页的能力，因此只能在用户理解其权限边界时启用。

页面完成检测

网页没有可供本地桥接器订阅的官方 `response.completed` 事件，不能只依据出现新文字就结束。最典型的错误是：刷新后页面暂时显示“Thinking”，桥接器把这个占位符当作最终回答返回；或者回答正文已经出现，但发送区域仍处于生成状态。

v0.2 综合使用以下信号：

- 生成中的停止按钮或忙碌状态；
- 输入框是否重新可用；
- 发送/语音控制是否恢复到空闲形态；
- 最新 assistant 文本是否持续稳定；
- 工具 YAML/JSON 是否形成完整闭合结构；
- 页面是否为网关错误或对话不存在，而不是普通回答。

判定原则： 可见的 Thinking、Working、Analyzing 等占位文本永远不是最终结果。页面必须回到空闲输入状态，且正文稳定或结构化工具块完整，才能完成本轮 Responses 请求。

若页面长时间没有进展，默认等待三分钟后刷新当前 URL；一次刷新后继续等待，不新建标签页，也不在结果未知时自动重发本轮提示。

长文本与工具协议

把十几 KB 的 developer 指令、工具 schema 和用户材料一次塞进 DevTools 调用，会增加转义错误、消息截断和连接重置概率。v0.2 的处理策略是：

- 保留工具名称、类型、必填参数和参数类型；
- 压缩重复说明、示例与 schema 噪声；
- 把过长 developer 提示替换为固定、可审计的代理规则摘要；
- 不截断用户消息和工具结果；
- 按块写入网页编辑器，并在页面内缓冲区校验总长度；
- 大型参考材料预留文件附件通道，而不是继续增大一次键盘注入。

网页模型必须返回一个 fenced YAML 工具调用或最终回答。YAML 的块文本比嵌套 JSON 更适合 PowerShell 命令，但模型仍可能偶尔把 `type: function_call` 写在代码围栏外，或在 Windows 路径、引号和换行处产生不合法结构。解析器会修复有限的反斜杠和引号问题；无法安全判断时则返回明确的 malformed tool call，而不是猜测后执行。

故障经验与对应设计

工作台、人工阶段与可选委托

Supervisor 把工作区、任务和事件长期保存。面板按讨论、规格确认、任务拆分、执行、GPT 复核和完成六个阶段从上到下展示。已经完成的阶段不会消失；用户切换项目后仍可从左侧找到不同工作区和历史任务。

阶段不会由模型自动推进。只有用户在当前对话明确同意后，`workflow_advance` 才能前进一层；系统也支持后退和暂停。这个约束把“模型认为可以继续”和“用户已经批准继续”分开。

DeepSeek V4 Flash 通过 OMP 作为可选微任务工人，不是默认主模型。一次委托只允许一个目标、一到两个文件、明确禁止路径和一条测试命令，并限制 diff 大小和重试次数。最终 diff 仍由主 GPT 检查并独立运行测试。

现象	原因或风险	v0.2 处理
每轮新开标签页	任务键或 URL 没有跨进程保存	持久化任务到 ChatGPT URL 映射。
/resume 后进入新网页对话	只恢复 Codex 历史，没有恢复网页会话身份	用同一任务键导航回旧 URL。
Thinking 被当成回答	只观察文字出现，没有检查编辑器空闲状态	占位符黑名单、控件状态与文本稳定联合判断。
504 后认为对话已坏	混淆网页请求失败与网页对话失效	退避并刷新同一 URL；只有硬失败阈值触发替代对话。
超长输入导致断流	一次 CDP 调用承载过多文本	分块注入、长度校验和精简 schema。
模型返回 malformed tool call	输出格式波动或代码围栏不完整	有限修复、白名单验证、失败显式返回并允许重新生成。
客户端重试导致重复工具	上次结果未知但实际已完成	请求指纹和已完成结果缓存。
固定 24 轮后换对话	人为阈值破坏网页上下文连续性	取消基于轮数和字符数的主动轮换。

表 5 真实调试现象如何转化为可测试的系统规则。

边界	当前措施	剩余风险
网络暴露	自建服务只监听 127.0.0.1	本机进程仍可能访问本地端口。
登录资料	不复制 Cookie 和浏览器 profile	附加标签页的调试权限本身很强。
公开仓库	排除密钥、真实对话、日志、截图、状态库和私用 ZIP	发布前仍必须执行凭据形态扫描。
工具权限	模型只返回白名单调用，实际由 Codex 执行	danger-full-access 下主 Codex 可访问工作区外文件和网络。
OMP 委托	文件清单、禁止路径、测试命令和 diff 限额	这不是操作系统级沙箱。
网页服务	不处理验证码、风控和用量限制	网页条款、UI 和可用性可能变化。

表 6 现有安全措施不是完整隔离，需要用户理解本机调试与 Agent 权限。

Exa MCP 写入隔离配置，用于需要时提供外部检索，但不会自动替代网页或 Codex 的工具链。

安全、隐私与信任边界

用户曾在聊天中粘贴过的 API key 不应进入代码或报告，应在服务商后台撤销并轮换。新的凭据通过 OMP 交互登录或 Windows DPAPI 本机加密方式保存，私用 ZIP 本身不得上传。

实现与验证

公开代码的主要模块如下：

- server.mjs: Responses、会话、缓存与检查点；
- chrome-mcp-browser.mjs: 已有 Chrome 控制与完成检测；
- tool-protocol.mjs: 工具声明、YAML/JSON 解析与校验；
- supervisor.mjs 与 state-store.mjs: 工作台服务和状态持久化；
- app-server-proxy.mjs: Codex App Server 事件复制；
- workflow-mcp.mjs 与 omp-worker.mjs: 人工阶段和受限委托；
- public/ 与 test/: 可视化界面和自动化测试。

2026-08-11 在 Windows 本地执行 npm run check 和 npm test。语法检查通过；52 项测试全部通过，0 项失败。测试覆盖见表 7。

这些测试证明当前代码中的协议和状态规则可重复执行，但不能证明 ChatGPT 网页长期维持相同 DOM，也

不能替代对真实 Chrome 权限、网络网关和模型格式波动的现场观察。

复现步骤

公开代码位于 [3099404236/chatgpt-web-codex-bridge](https://github.com/3099404236/chatgpt-web-codex-bridge)。在 Windows PowerShell 中：

```
cd code
npm install
./scripts/install-codex-web.ps1
codex-web doctor
```

然后在任意项目目录运行 codex-web。关闭终端不会删除 Codex 任务；再次启动后可在 TUI 中使用 /resume。工作台地址为 <http://127.0.0.1:8770/>。

后续研究方向

当前检查点是工程上的可解释替代物。更进一步的问题是：当开源模型允许读取 KV Cache、隐藏激活和显式推理 token 时，能否把巨大、模型专属的内部状态压缩为小型、可审计、跨模型可迁移的 Agent checkpoint？

一个可验证的研究设计是，在长工具任务中随机注入进程关闭、网页断流、模型切换和上下文截断，比较完整历史、最近窗口、自然语言摘要、结构化检查点、KV Cache 与隐藏状态压缩。指标包括任务成功率、重复工具次数、恢复轮数、状态大小、约束遗漏、错误事实写入率和跨模型迁移能力。当前桥接器可以作为黑盒故障环境，开源权重模型则提供白盒参考上限。

测试组	结果	主要覆盖
Responses 与 SSE	通过	普通回答、函数调用、自定义补丁、完成事件和失败终态。
会话恢复	通过	跨重启 URL 恢复、暂态失败保留 URL、硬失败检查点交接。
去重与增量	通过	相同完成请求缓存、前缀匹配和最近可恢复增量。
网页控制	通过	Chrome MCP 页面解析、长文本分块、空闲发送控件和错误页识别。
工具协议	通过	白名单、YAML/JSON、Windows 路径与引号修复、malformed 拒绝。
补丁边界	通过	新增、更新、移动、删除与工作区外路径拒绝。
工作台状态	通过	工作区、任务、人工门控、App Server 事件与持久化。

表 7 v0.2 的自动化验证范围。测试套件不包含真实账号或真实 ChatGPT 对话。

局限与结论

v0.2 解决的是“如何更稳地复用与恢复”，不是“如何把网页变成稳定原生 API”。它仍受网页 DOM、浏览器权限、网络质量、模型输出格式和产品条款影响。结构化检查点能保留目标、工具 证据和同步位置，但不能复制官方 persisted reasoning，也不能保证替代对话与原对话具有 完全相同的推理连续性。

第二版的主要结论是：对网页型 Agent 而言，连接失败不应等同于会话失效；请求完成检测、会话身份、工具幂等和恢复检查点必须分开建模。只有这样，刷新、断流和终端关闭才不会自然 演化成新标签页、重复输入和重复工具执行。

参考文献

1. OpenAI. Conversation State. <https://developers.openai.com/api/docs/guides/conversation-state> (2026 年).
2. OpenAI. Function Calling. <https://developers.openai.com/api/docs/guides/function-calling> (2026 年).
3. OpenAI. Compaction. <https://developers.openai.com/api/docs/guides/compaction> (2026 年).
4. OpenAI. Reasoning Models: Preserve Reasoning Across Calls. <https://developers.openai.com/api/docs/guides/reasoning> (2026 年).
5. ChromeDevTools chrome-devtools-mcp contributors. autoConnect fails on Chrome 150 stable default profile: port 9222 listens but /json/version returns 404, DevToolsActivePort never created. <https://github.com/ChromeDevTools/chrome-devtools-mcp/issues/2283> (2026 年).